Police Shootings Analysis

A Brief: what it does, what it gets right, and what is broken

Explainer for `police-shootings-analysis`

In one paragraph

`police-shootings-analysis` reads five fixed CSV files and turns them into charts, two different ways. A single 392-line script, `analyse_deaths.py`, writes 24 static chart files and exits. A small Flask web app, `app.py`, serves three of the same views as an interactive dashboard that filters by state, year, and armed status. It began as a guided Udemy exercise about joining messy real-world data and comparing three plotting libraries. The dashboard is later, better work built beside it. The two halves share one CSV file on disk and essentially no code.

The subject, and the headline finding

Every row of the main dataset is a person who was killed. This document treats that data descriptively and states its limits rather than repeating chart titles on trust.
The single most important thing to know about this project: **three of its 24 charts are titled "High School Graduation Rate by State" and actually plot the number of cities per state**, on a y-axis running to 1750. One of them is committed to the repository as a PNG. The cause is `.count()` where `.mean()` was meant. The fix is one word. It is covered in the "What is broken" section, and in full in the Deep Dive.

1 What the data can and cannot say

What it is	What that means
A fixed extract of The Washington Post's fatal-police-shootings database: 2,535 records, 2015 through part of 2017.	Not live, not current, and the code has no way to fetch an update. A chart titled "Killings by Year" stops in 2017 and cannot know that.
Four city-level Census snapshots from around 2015 (poverty, high school completion, race share, income).	Static too. And the fatality data is never actually joined to any of them, despite the README's framing.
Counts of *records*.	Not counts of events. Anything that shaped what got reported is invisible here.
Raw totals with no population denominator.	The map and the "top 10 cities" chart substantially rank by population. California leads at 424 records and is the most populous state. A code comment asks "which cities are the most dangerous"; the chart cannot answer that.
Correlations.	Never causation. The strongest relationship in the project, city poverty against city high school completion, is about −0.50, verified. That is an association and nothing more.
Some very small cells.	The race-by-top-10-city chart's bars hold between 10 and 21 records each, verified. Nothing stable survives at that size.

2 Architecture

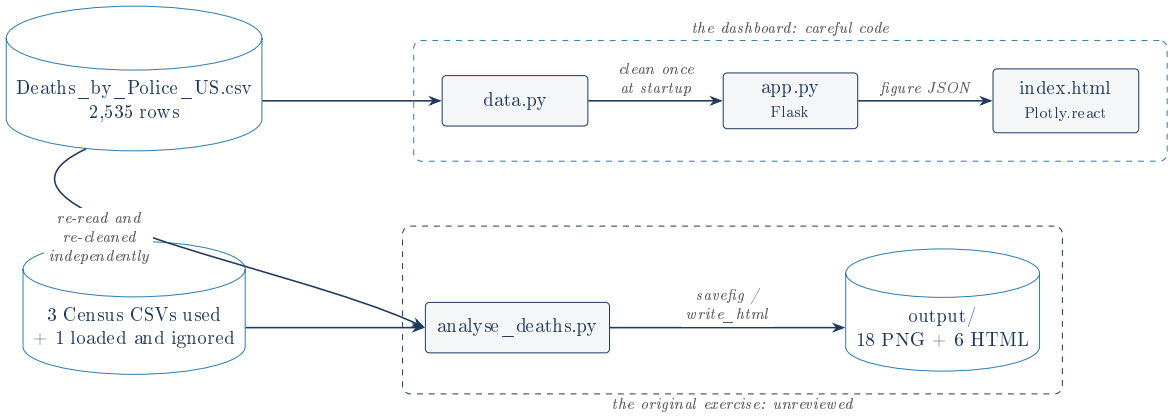

Figure 1: Two entry points sharing a file, not a code path. The curved arrow is the whole problem: `analyse_deaths.py` does not import `data.py`, so the two halves keep duplicate copies of the same cleaning logic and can drift apart.

The shape to remember

`data.py` is described in its own docstring as the single home of the cleaning logic. It is the single home of that logic *for the dashboard*. `analyse_deaths.py` still has its own inline copy and imports nothing. The abstraction was extracted for the new code; the old code was left pointing at a duplicate.

2.1 The static script

Straight-line code, no classes, two small functions. Four decisions define it:

1. `matplotlib.use('Agg')` on line 2, before `pyplot` is imported. This is the single most valuable change made to the original notebook exercise: it makes the script run anywhere, with no display, writing durable files instead of opening windows that never appear.
2. A global counter, `next_chart_path()`, names every output `chart_NN`. The numbering is pure execution order, so the PNGs are only meaningful alongside the exact script version that made them, and nothing enforces that pairing.
3. `fillna(0)` on all five dataframes, before any numeric coercion. This one is a bug and is covered below.
4. The same chart drawn in matplotlib, seaborn, and plotly, which is the actual point of the exercise.

Charts 01–12 come from the Census data. Charts 13–24 come from the fatality data. The two groups never meet.

2.2 The dashboard

`data.py` loads and cleans once and exposes `apply_filters()`. `app.py` holds the cleaned dataframe in memory at import time, and on each request filters it and rebuilds three Plotly figures (a choropleth, a year bar chart, a top-10-cities bar chart) plus a four-value stat row. `templates/index.html` server-renders the first payload, then calls `GET /api/charts` on every filter change and updates the charts in place with `Plotly.react`, so hover and zoom survive and nothing is a static image. Clicking a state on the map sets the state dropdown and re-fetches, so the map and the dropdown cannot disagree.

3 Verified by running it

Everything here came from a clean virtual environment built from `requirements.txt`, not from the README.

Claim	Result	Verdict
2,535 records; 2,428 M / 107 F	as stated	confirmed
633 of 2,535 with signs of mental illness	25.0%	confirmed
Top armed: gun 1,398, knife 373, vehicle 177	as stated	confirmed
Top states: CA 424, TX 225, FL 154	as stated	confirmed
24 charts produced (18 PNG, 6 HTML, 31 MB)	as stated	confirmed
Dashboard, including the zero-row filter case	renders, shows n/a	confirmed
Killings per year	**991 / 963 / 581**, not 991 / 962 / 582	**README wrong**
"regenerates in under a minute"	**6 min 54 s**	**README wrong**

4 What is broken

1. Three charts plot the wrong quantity (charts 04, 05, 06)

`groupby('Geographic Area')['percent_completed_hs'].count()` counts rows per state. It never looks at the column's values. `.mean()` is what a rate needs.
So three charts titled "High School Graduation Rate by State" plot how many cities each state has. Rendered and read: the y-axis reaches 1750, the tallest bars are PA 1,761, TX 1,710, CA 1,501, and the shortest is DC at 1, because DC contributes one city row. Chart 04 is committed to the repository.
The tell was available: chart 07, in the same script, plots the same quantity correctly via seaborn's implicit mean, topping out near 100. Two charts in one document disagree by a factor of 17. Nothing noticed, because the project has no tests at all.

2. The income dataset is loaded and never used, and nothing is cross-referenced

The README's first sentence says the fatality data is "cross-referenced against four US Census datasets ... to look for socioeconomic patterns".
`df_hh_income` appears twice in the whole project: line 37 reads the 709 KB file, line 46 calls `fillna(0)` on it. Nothing reads it again. Verified by grep.
The larger version, which an interviewer will reach: **the fatality data is never joined to any Census data at all**. The Census charts and the fatality charts are two separate analyses sharing a script. The cross-reference is an aspiration, not a feature.

3. `fillna(0)` runs before the coercion meant to catch bad values

Because missing cells become the number 0 first, the `errors='coerce'` that was supposed to turn them into NaN never sees them. Verified consequences: **77 records with no recorded age are plotted as zero-year-olds** in the age histogram and KDE (chart 17), pulling the density curve left with no visual cue that the points are fabricated; and **195 records with no recorded race become a bar literally labelled** `0` in chart 18.
The inline comment above those lines reads "Replace 0 with NaN". The code does the exact opposite, and the two operations have opposite consequences.

`data.py` inherited the same inverted order, so both halves share it. In the dashboard's favour: `app.py` and `data.py` both explicitly strip the `0` values at the point of use, so the author did notice the symptom there and patched it locally rather than at the source. The script never got that treatment.

4. Chart 12 is not stacked, and the README says it is

The loop draws five bar series at `alpha=0.7` and never passes `bottom=`, so all five start at zero and overplot each other. Rendered and confirmed: the White bar covers almost everything drawn after it. A reader who expects a stacked chart reads a bar at 92 as a cumulative total when it means "the White share is 92".
The underlying arithmetic is also an unweighted mean across cities, so a village counts as much as a city of 800,000. Defensible, but never acknowledged.

5. `scatter={'alpha': ...}` does nothing (charts 10, 11)

Verified against the installed seaborn 0.13.2 signature: `regplot` and `lmplot` take `scatter` as a **boolean**, and `scatter_kws` as the styling dict. A non-empty dict is truthy, so the alpha is silently discarded and no error is raised. The author knew the right pattern: `line_kws={'color': 'red'}` sits right beside it and works correctly. So two scatter plots of 29,329 points render at full opacity when transparency was intended.

4.1 Smaller ones

- `convert_country_code` **is a guarded no-op**, kept alive purely to support chart 23, a world-scope map plotting US state postal codes onto countries. Chart 23 is not meaningful and `pycountry` exists in `requirements.txt` only for it. (That some state codes collide with real ISO country codes and are therefore silently rewritten follows from `pycountry`'s contract but was **not measured** on this dataset: treat it as inferred.)
- **Chart 01 does not aggregate.** matplotlib is handed 29,329 city rows and draws them all onto 50 x-positions. Chart 02, the seaborn version of the same title, averages per state. Two charts, same title, different quantities.
- `GET /` **ignores its query string**, so a filtered view has no URL and cannot be bookmarked or shared. `/api/charts` already parses exactly those parameters correctly.
- **The Plotly CDN** is an unpinned single point of failure with no integrity hash, so a local dashboard over local data needs the internet to draw anything.
- `fetchAndRender` **has no** `catch`, so a failed request leaves stale charts on screen that no longer match the dropdowns.
- **No tests and no schema validation.** This is the root cause of items 1 through 5.

5 What it gets right

- The headless `Agg` backend plus file output is what makes the script reproducible at all. It was a real fix to a real problem.
- `base_path` from `sys.argv[0]`: the script finds its data from any working directory.
- Exact version pins on every dependency, including `scipy`, which the project *never imports* and cannot run without: both KDE charts delegate to `scipy.stats.gaussian_kde` lazily at draw time. Declaring a behavioural dependency the import graph does not show is exactly right.

- `format='mixed'` on the date column, which genuinely mixes `M/D/YYYY` and `DD/MM/YY`. Verified to parse all 2,535 rows with zero failures.
- `fig_to_json` routes Plotly figures through Plotly's own encoder, sidestepping a real Flask/numpy serialization failure before it could happen at request time.
- `build_stats` handles the empty selection. Tested: it returns nulls and the page shows "n/a" instead of raising `IndexError`.
- The `# noqa: E712` on `== True` is the author correctly overruling the linter: on a pandas Series that is a vectorised comparison and `is True` would be wrong.
- Map clicks route *through* the dropdown rather than around it, so two controls over one piece of state cannot disagree.
- The README's "Data and framing notes" and the dashboard footer both state the causation and sample-size limits unprompted. On this subject that matters, and most exercise write-ups skip it.

6 The fix list

1. `.count()` to `.mean()` at line 89. One word, fixes three false charts.
2. Move `fillna(0)` after the coercions, or delete it. Fixes charts 17 and 18, in both entry points.
3. Use `df_hh_income` or delete it, and rewrite the README's "four Census datasets … cross-referenced" to describe what the code does.
4. Pass `bottom=` in the chart 12 loop, or stop calling it stacked.
5. `scatter=` to `scatter_kws=` in charts 10 and 11.
6. Delete `convert_country_code`, chart 23, and `pycountry`.
7. Correct the README's year counts and its runtime claim.
8. Give each chart a function and a name; add one test that asserts a rate chart's maximum is under 100.

The defence, in one line

The dashboard half is careful, well-factored code with real thought in it. The static-script half is an exercise that was made to run headlessly and then never looked at again, and it contains three charts that are confidently, visibly false. The gap between the two is the whole story, and the reason for the gap is that neither half has a single test.

Glossary

CSV
: Plain-text table, commas between columns. Carries no type information, which is why a column of numbers arrives as text and has to be coerced.

Character encoding
: The byte-to-character mapping. All five files here are Windows-1252, not UTF-8, and three of them use bare carriage returns as line endings.

DataFrame
: pandas' in-memory table. The unit of work for everything in this project.

NaN / NaT
pandas' "missing number" and "missing date". Deliberately contagious, so that missingness forces a decision instead of quietly becoming a plausible number.

Coercion
`pd.to_numeric(col, errors='coerce')`: convert to a number, turn anything unconvertible into NaN rather than crashing.

groupby
Split a table by a key, compute per group, reassemble one row per group.

Backend (matplotlib)
What matplotlib draws onto. `Agg` draws into memory instead of a window, which is what makes the script runnable with no display.

KDE
Kernel Density Estimate: a smooth curve estimating a distribution's shape. A smoothed guess, not a measurement, and it will smooth fabricated data into what looks like a real feature.

Choropleth
A map that colours regions by a value. Colours by count here, not rate.

Correlation
A number in $[-1, +1]$ for how tightly two quantities move together. Says nothing about why.

Flask
A minimal Python web framework. Maps URLs to functions.

Jinja2 / `| safe`
Flask's template language, and the filter that disables its automatic HTML escaping. Necessary for injecting pre-built JSON; safe here only because the data is trusted and static.

Serialization
Turning an in-memory object into bytes for the network. JSON knows few types, so numpy integers need an explicit rule.

Transitive dependency
A package installed because something else needed it. Relying on one without declaring it, as this project would with `scipy`, breaks the day the intermediate drops it.

CDN
A third-party host serving a library over the internet. Convenient, and a runtime dependency on someone else's uptime.